

Task 16 —: Conduct an AWS IAM Security Review

Intern: Nidhi Pandey

Program: Evolixa AWS Cloud Internship

Date: 2026-09-05

1. Approach and Architecture

The objective of this task was to perform a structured review of a fictional AWS account's IAM configuration and identify excessive or poorly designed permissions. The security team wanted to answer one critical question: "Who has access to what, and where are we giving more access than necessary?"

My approach was to systematically inventory all IAM identities, map users and roles to their policies, identify administrator-level and wildcard permissions, and find unused or unnecessary access. I then compared the permissions with job responsibilities, built an access matrix, proposed least-privilege improvements, and created a remediation plan. Finally, I prepared a professional security review report summarizing all findings and recommendations.

2. Implementation Steps

Step 1 – Inventory the IAM Identities: I started by listing all IAM identities in the account. This included users, roles, and policies. I documented each identity along with its purpose and attached policies.

Step 2 – Map Users/Roles to Their Policies: I mapped each IAM user and role to the policies attached to them. This gave me a clear picture of who had access to what.

Step 3 – Identify Administrator-Level Permissions: I identified users and roles that had AdministratorAccess or similar broad permissions. These were flagged as high-risk.

Step 4 – Identify Wildcard Permissions: I reviewed policies for wildcard permissions like "Action": "*" or "Action": "s3:*". These permissions are too broad and should be restricted.

Step 5 – Identify Unused or Unnecessary Access: I checked for users with no recent activity and roles that were not being used. These were flagged for cleanup.

Step 6 – Compare Permissions with Job Responsibilities: I compared the actual permissions of each user with what they should have based on their job role. This helped identify over-privileged users.

Step 7 – Build an Access Matrix: I created an access matrix to visualize the permissions of each user across different services like EC2, S3, IAM, and Billing.

Step 8 – Identify Potential Least-Privilege Improvements: I identified permissions that were too broad and proposed more restrictive policies.

Step 9 – Propose Revised Policies: I wrote revised policies that followed the principle of least privilege, restricting actions to only those required for the job.

Step 10 – Test the Revised Permissions in a Controlled Environment: I tested the revised policies using a test user to confirm that allowed actions worked and denied actions were properly blocked.

Step 11 – Document Allowed and Denied Actions:

I documented the results of the permission tests, listing which actions were allowed and which were denied.

Step 12 – Prioritize the Security Findings: I prioritized findings based on their impact on security and compliance.

Step 13 – Explain the Potential Impact of Excessive Permissions: I documented the risks associated with excessive permissions, such as data breaches and resource deletion.

Step 14 – Create an IAM Remediation Plan: I created a plan with specific actions, timelines, and owners to address the identified issues.

Step 15 – Prepare a Professional Security Review Report: I compiled all findings, recommendations, and the remediation plan into a professional security review report.

3. Key AWS Concepts Learned

- **IAM Inventory:** I learned the importance of maintaining a complete inventory of all IAM identities to understand the security posture of an AWS account.
- **Least Privilege:** I gained a deeper understanding of the principle of least privilege and how to apply it in practice.
- **Wildcard Permissions:** I learned to identify and avoid wildcard permissions that grant broad, unnecessary access.
- **Access Matrix:** I understood how to build an access matrix to visualize permissions across users and services.

- **Permission Testing:** I practiced testing permissions using the AWS CLI to confirm that policies were correctly enforced.
 - **Remediation Planning:** I learned how to create a structured remediation plan with priorities and timelines.
 - **Security Reporting:** I gained experience in preparing professional security reports suitable for management review.
-

4. Challenges Faced

- **Identifying Unused Users:** The biggest challenge was identifying which users were truly unused. Some users had no recent activity but might still be needed for specific tasks.
 - **Mapping Policies to Users:** Mapping each user to their policies was time-consuming because I had to check each user's permissions individually.
 - **Understanding Job Responsibilities:** I had to make reasonable assumptions about what each user should have access to based on their job role.
 - **Testing Denied Actions:** Testing denied actions required careful command execution to ensure that the permissions were correctly restricted.
 - **Prioritizing Findings:** Prioritizing findings was challenging because I had to balance security impact with operational impact.
-

5. Solutions Implemented

- **Systematic Inventory:** I created a systematic approach to inventory all IAM identities, using the AWS Console and CLI to gather information.
 - **Access Matrix:** I built an access matrix to visualize permissions and identify gaps or overlaps.
 - **Least-Privilege Improvements:** I proposed revised policies that followed the principle of least privilege, restricting actions to only those required for the job.
 - **Permission Testing:** I tested the revised policies using a test user to confirm that allowed actions worked and denied actions were properly blocked.
 - **Remediation Plan:** I created a structured remediation plan with priorities, timelines, and owners to address the identified issues.
 - **Security Report:** I compiled all findings and recommendations into a professional security review report.
-

6. Security Considerations

- **Excessive Permissions:** I found that the task3-iam-user had AdministratorAccess, which is a high-risk permission. This user should have only the permissions required for their role.
- **Wildcard Permissions:** I identified policies with wildcard permissions like "Action": "*" and "Action": "s3:*". These should be replaced with more specific permissions.
- **Unused Users:** I found users with no recent activity, such as hr-admin-user, mytask, and operations-user. These should be deleted or disabled.

- **Missing MFA:** I noted that no IAM users had MFA enabled, which is a significant security gap.
 - **Missing CloudTrail:** I noted that CloudTrail was not enabled, which meant API calls were not being logged for audit purposes.
 - **Principle of Least Privilege:** I emphasized the importance of following the principle of least privilege to reduce the risk of data breaches.
-

7. Results and Observations

Inventory Summary:

I identified nine IAM users, one custom IAM role, and five customer-managed policies. The users included developer-user, qa-user, s3-admin-user, task3-iam-user, and several others.

Excessive Permissions:

I found that task3-iam-user had AdministratorAccess, which grants full access to all AWS services and resources. This is a high-risk permission that should be removed. Additionally, s3-admin-user had S3FullAccessPolicy, which allows full access to all S3 buckets.

Wildcard Permissions:

Both AdministratorAccess and S3FullAccessPolicy contained wildcard permissions. AdministratorAccess had "Action": "*", and S3FullAccessPolicy had "Action": "s3:*". These should be replaced with more specific permissions.

Unused Users:

I identified four users with no recent activity: hr-admin-user, mytask, operations-user, and task3-iam-user. These users should be reviewed and either deleted or disabled.

Access Matrix:

The access matrix showed that developer-user and qa-user had appropriate permissions, while s3-admin-user had broader permissions than needed, and task3-iam-user had full administrative access.

8. Possible Improvements for Production

If this environment were in production, I would recommend the following improvements:

- **Least-Privilege Policies:** Replace broad policies like AdministratorAccess and S3FullAccessPolicy with least-privilege policies that grant only the permissions required for each role.
- **MFA Enforcement:** Enable MFA for all IAM users to add an extra layer of security.
- **CloudTrail Enablement:** Enable CloudTrail to log all API calls for audit and compliance purposes.
- **Regular Permission Reviews:** Schedule quarterly reviews of all IAM permissions to remove unnecessary access.
- **IAM Identity Center:** Use IAM Identity Center for centralized user management and single sign-on.
- **Permission Boundaries:** Use permission boundaries to limit the maximum permissions a user can have.

- **Automated Access Reviews:** Implement automated access reviews using AWS IAM Access Analyzer.
-

9. Remediation Plan

The following remediation plan was created to address the identified security issues:

- **Remove AdministratorAccess from task3-iam-user:** This is a high-priority action that should be taken immediately. The user should be given only the permissions required for their role.
 - **Restrict s3-admin-user to Specific S3 Buckets:** The S3FullAccessPolicy should be replaced with a policy that restricts access to only the required S3 buckets.
 - **Delete or Disable Unused Users:** Users with no recent activity, such as hr-admin-user, mytask, and operations-user, should be deleted or disabled.
 - **Enable MFA for All Users:** MFA should be enabled for all IAM users to improve security.
 - **Enable CloudTrail:** CloudTrail should be enabled to log all API calls for audit and compliance purposes.
 - **Review IAM Policies:** All IAM policies should be reviewed to ensure they follow the principle of least privilege.
-

10. Security Review Report

markdown

IAM Security Review Report

Executive Summary

An IAM security review was conducted on the fictional AWS account to identify excessive permissions, unused users, and security gaps. The review found that `task3-iam-user` had AdministratorAccess, several users were unused, and MFA and CloudTrail were not enabled.

Key Findings

1. ****Excessive Permissions:**** `task3-iam-user` had AdministratorAccess, granting full access to all AWS services and resources.
2. ****Wildcard Permissions:**** Policies contained wildcard permissions like `"Action": "\*"` and `"Action": "s3:\*"`.
3. ****Unused Users:**** Four users had no recent activity and should be deleted or disabled.
4. ****Missing MFA:**** No users had MFA enabled.
5. ****Missing CloudTrail:**** CloudTrail was not enabled for audit logging.

Recommendations

1. Remove AdministratorAccess from `task3-iam-user`.
2. Restrict `s3-admin-user` to specific S3 buckets.
3. Delete or disable unused IAM users.

4. Enable MFA for all IAM users.
5. Enable CloudTrail for audit logging.
6. Review all IAM policies to follow the principle of least privilege.

Remediation Plan

Action	Timeline	Owner
-----	-----	-----
Remove AdministratorAccess	Immediate	Security Team
Restrict S3 permissions	1 week	Security Team
Delete unused users	1 week	Security Team
Enable MFA	2 weeks	All Users
Enable CloudTrail	2 weeks	Security Team

Conclusion

This IAM security review identified several critical issues, including excessive permissions, unused users, and missing security controls. By following the recommended remediation plan, the organization can significantly improve its security posture and reduce the risk of data breaches. The key takeaway is that regular IAM reviews are essential for maintaining a secure AWS environment.